
`github.com/mg-i-roc-maz/corsi · microservizi/lezione-1`

LEZIONE 1 DI 8 · 3 ORE

Dal monolite ai microservizi

Cosa sono, quanto costano, quando non usarli e come si trovano i confini tra i servizi.

Rocco Mazzeo

`rocco.mazzeo@gmail.com`

Fabio Rillo

`fabiorillo.work@gmail.com`

Le tre ore di oggi

Teoria · 1h15

Teoria

Monolite e microservizi, vantaggi e costi, quando non usarli, bounded context.

Laboratorio · 1h35

Laboratorio

Esplorare il monolite, costruire il servizio catalog, checkpoint con npm test.

Ultimi 10'

Una domanda per chiudere: «cosa si rompe se...?»

IL PUNTO DI PARTENZA

**Un negozio online deve mostrare i prodotti,
ricevere gli ordini e avvisare i clienti.**

Domanda: quanti programmi servono per farlo? Uno solo, o uno per ogni compito?

Il monolite

Un'unica applicazione che fa tutto: un processo, un deploy, un database.

Dentro può essere ben organizzato in moduli, ma si avvia, si aggiorna e si rompe tutto insieme.



Monolite non vuol dire disordinato

DA EVITARE

Monolite «a spaghetti»

In inglese *big ball of mud*. Ogni parte usa direttamente qualunque altra: cambiare il catalogo può rompere gli ordini in modi imprevedibili.

DA CERCARE

Monolite modulare

Un solo processo, ma diviso in moduli con confini chiari: ogni modulo espone poche funzioni e nasconde il resto.

Il monolite modulare è spesso il punto di partenza migliore, e il passo intermedio più sicuro prima dei microservizi.

Perché il monolite funziona

<> Semplice da sviluppare

Un repo, un progetto da aprire, una funzione chiama l'altra.

▶ Un solo deploy

Si copia un pacchetto su un server e il sistema è aggiornato.

🗄️ Dati coerenti

Una transazione aggiorna ordine e scorte insieme, o niente.

🔍 Debug immediato

Un breakpoint e uno stack trace mostrano tutto il percorso.

Quando il monolite inizia a pesare

Team che si pestano i piedi

Venti persone sullo stesso codice: conflitti, attese, revisioni infinite.

Deploy tutto o niente

Per correggere un testo nel catalogo si rilascia anche il codice degli ordini.

Si scala tutto insieme

Il catalogo riceve cento volte più richieste degli ordini, ma si duplica l'intera applicazione.

Un guasto ferma tutto

Un errore nelle notifiche manda in crash il processo: nessuno vede più i prodotti.

Scalare: le tre dimensioni

Lo **Scale Cube** di Martin Abbott e Michael Fisher (*The Art of Scalability*, 2009) distingue tre modi di reggere più carico.

Asse	Cosa si fa	Nel negozio
X · clonare	Più copie identiche dell'applicazione	Tre copie del monolite
Y · scomporre	Parti diverse per funzioni diverse	catalog, orders, notifications
Z · partizionare	Ogni copia gestisce una parte dei dati	Clienti A–M e N–Z separati

I microservizi sono l'**asse Y**. Un monolite scala già lungo X, e spesso basta.

I microservizi

Tante piccole applicazioni, ognuna responsabile di una parte del dominio, che collaborano attraverso la rete.

Focalizzati

Un compito, fatto bene.

Indipendenti

Ognuno ha il suo deploy.

Con i loro dati

Nessuno legge il database di un altro.

In rete

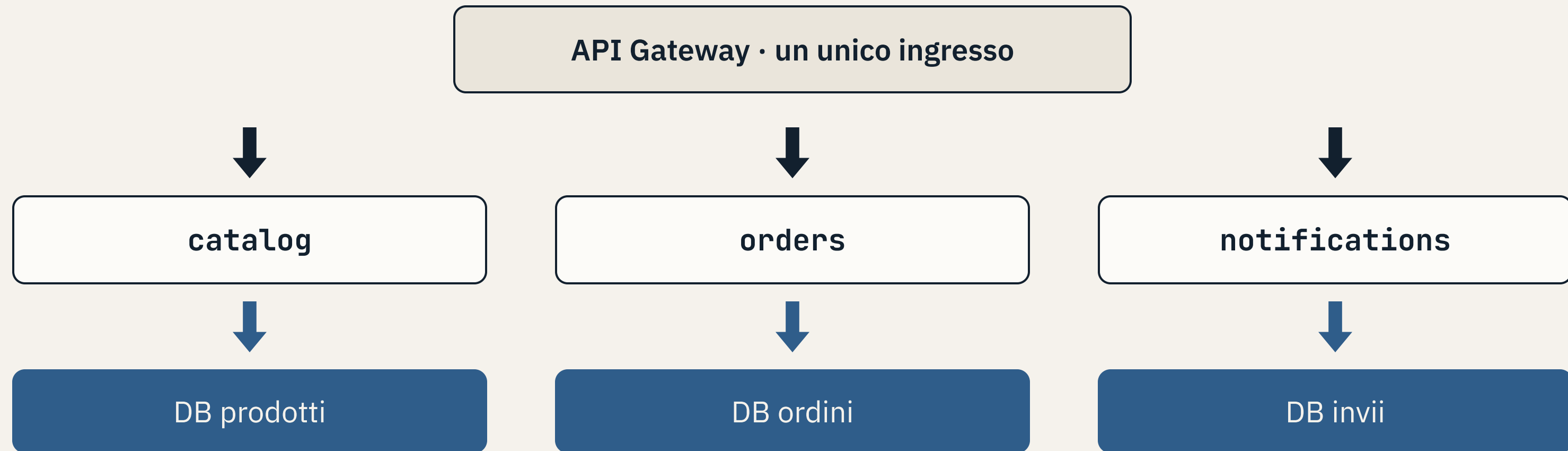
Parlano con HTTP o messaggi.

Le caratteristiche dei microservizi

Dall'articolo *Microservices* di James Lewis e Martin Fowler (2014), il riferimento classico.

Caratteristica	In pratica
Organizzati per capacità di business	Un servizio per «il catalogo», non per «il database»
Deploy indipendente	Si aggiorna catalog senza toccare orders
Dati decentralizzati	Ogni servizio ha il suo database, solo suo
Endpoint intelligenti, canali semplici	La logica sta nei servizi, la rete trasporta
Progettati per il guasto	Funzionano anche se un altro servizio non risponde
Infrastruttura automatizzata	Build, test e rilasci senza passaggi manuali

Lo stesso negozio, a microservizi



Tre processi, tre deploy, tre database. Ognuno si può aggiornare, scalare o spegnere da solo.

Che cosa si guadagna



Team autonomi

Ogni team possiede i suoi servizi e li rilascia con i suoi tempi.



Rilasci piccoli

Un rilascio tocca un solo servizio: rischio contenuto, rilasci frequenti.



Scala mirata

Si moltiplicano solo i servizi sotto carico.



Guasti isolati

Se notifications cade, si può ancora vedere e ordinare.



Tecnologie diverse

Ogni servizio può usare lo strumento più adatto, con giudizio.



Ma...

Ogni vantaggio ha un costo. Vediamolo.

Monolite e microservizi a confronto

Aspetto	Monolite	Microservizi
Deploy	Uno, tutto insieme	Uno per servizio
Scalabilità	Si replica tutto	Si replica solo ciò che serve
Dati	Un database, transazioni	Un database per servizio
Guasti	Un errore ferma tutto	Si degrada una parte
Chiamate	Funzioni, in memoria	Rete: lente, possono fallire
Debug	Uno stack trace	Log sparsi su più servizi
Infrastruttura	Un server	Container, gateway, monitoraggio

I costi che non si vedono subito

La rete non è una funzione

Le otto «fallacie del calcolo distribuito» (L. Peter Deutsch e colleghi, Sun Microsystems): ipotesi che sembrano vere e sono false.

- | | |
|------------------------|---------------------------|
| 1 la rete è affidabile | 5 la topologia non cambia |
| 2 la latenza è zero | |
| 3 la banda è infinita | 6 un solo amministratore |
| 4 la rete è sicura | 7 trasporto a costo zero |
| | 8 la rete è omogenea |

E poi c'è il resto

Consistenza dei dati

Niente transazione unica tra servizi.

Osservabilità

Seguire una richiesta tra più processi.

Infrastruttura e competenze

Container, gateway, CI/CD, monitoraggio.

Quanto costa una chiamata

~ns

Funzione in memoria

Miliardesimi di secondo. Non fallisce per colpa della rete.

~0,5 ms

Andata e ritorno in rete

Anche nello stesso datacenter. Può essere lenta, perdersi, fallire a metà.

Centinaia di migliaia di volte di più. Dieci chiamate in sequenza: i ritardi si sommano.

Niente più transazione unica

MONOLITE

```
BEGIN  
salva ordine  
scorte -1  
COMMIT
```

Riesce tutto o non cambia niente.

MICROSERVIZI

```
orders: salva ordine ✓  
orders → catalog: scorte -1  
catalog: ...nessuna risposta
```

L'ordine c'è, le scorte non sono scese. E adesso?

Si accetta la **consistenza eventuale**: i dati diventano coerenti dopo un po', e si progetta come gestire i fallimenti a metà.

Quando NON usarli

- ⚠ Il team è piccolo, uno solo
- ⚠ Il dominio non è ancora chiaro
- ⚠ È un prodotto nuovo, da validare
- ⚠ Mancano automazione e monitoraggio

MONOLITH FIRST

Partire da un monolite ben diviso in moduli e separare un servizio solo quando il suo confine è chiaro e il vantaggio è concreto.

Idea resa popolare da Martin Fowler nell'articolo «MonolithFirst» (2015).

Casi reali, nelle due direzioni

VERSO I MICROSERVIZI

Netflix

Dopo un grave guasto del database nel 2008 migra verso cloud e microservizi; completa nel 2016. Centinaia di team: i vantaggi arrivano.

RITORNO

Segment

Nel 2018 (*Goodbye Microservices*) riunisce centinaia di servizi quasi identici: costavano più di quanto rendevano.

RITORNO

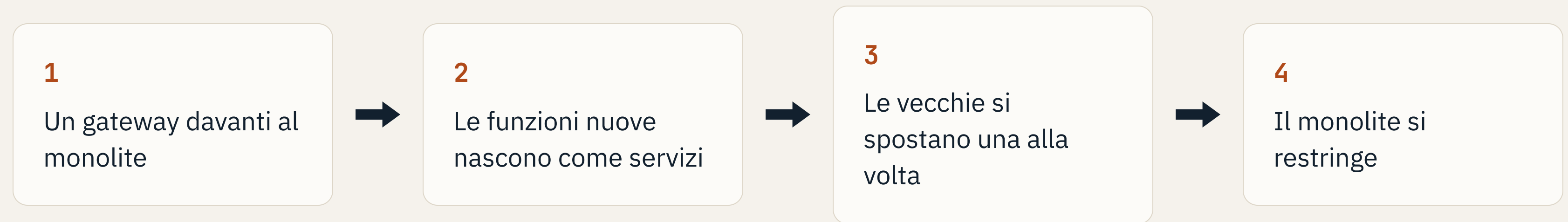
Prime Video

Nel 2023 riporta in un'unica applicazione un servizio di monitoraggio distribuito: costi infrastrutturali –90%.

La scelta dipende dal problema, e può cambiare nel tempo.

Migrare un pezzo alla volta

Lo **Strangler Fig pattern** (Martin Fowler): come il fico strangolatore cresce attorno a un albero, i servizi crescono attorno al monolite finché lo sostituiscono.



LA LEGGE DI CONWAY, IN SINTESI

L'architettura di un sistema tende a ricalcare il modo in cui comunicano i team che lo costruiscono.

Formulata da Melvin Conway nel 1967. Per questo i microservizi sono prima di tutto una scelta organizzativa: un servizio, un team che lo possiede.

Bounded context

Un confine dentro il quale ogni parola del dominio ha un significato preciso e un solo modello la rappresenta.

Da dove viene

Domain-Driven Design, Eric Evans (2003).

A cosa serve

A decidere dove tagliare il sistema.

La regola pratica

Un bounded context è un buon candidato per un servizio.

Che cos'è un «prodotto»?

PER IL CATALOGO

Cosa si vende

nome
descrizione
prezzo di listino
foto

PER GLI ORDINI

Cosa è stato comprato

id prodotto
quantità
prezzo pagato
nome al momento

PER IL MAGAZZINO

Cosa c'è sugli scaffali

scorte
ubicazione
fornitore
riordino

Stessa parola, tre modelli diversi. Forzarli in un'unica tabella **prodotti** è il primo segnale di un confine mancante.

Quattro domande per trovare i confini

1 Cambiano insieme?

Codice che cambia sempre insieme sta dalla stessa parte.

2 Chi possiede i dati?

Ogni dato ha un solo servizio che può modificarlo.

3 Le parole cambiano?

Se un termine cambia significato, lì c'è un confine.

4 Un team può possederlo?

Se serve coordinare tre team per ogni modifica, il taglio è sbagliato.

Non tutti i domini valgono uguale

CORE

Ciò che ci distingue

Catalogo e raccomandazioni. Qui si investe il meglio del team.

SUPPORTING

Necessario

Gestione ordini. Va fatto bene, ma non è ciò che fa scegliere il negozio.

GENERIC

Uguale per tutti

Email, pagamenti. Spesso conviene un servizio esterno.

I nostri tre servizi

Servizio	Si occupa di	Possiede	Non fa
catalog	Prodotti in vendita	Prodotti, prezzi	Non crea ordini
orders	Ricevere gli ordini	Ordini e righe	Non modifica i prodotti
notifications	Avvisare i clienti	Modelli, invii	Non decide quando avvisare

La colonna **Non fa** conta quanto le altre: un confine si definisce anche da ciò che resta fuori.

Anti-pattern: il monolite distribuito

Database condiviso

Tutti i servizi leggono le stesse tabelle: cambiare una colonna rompe tutti.

Chiamate a catena

A chiama B che chiama C che chiama D: se uno cade, cadono tutti.

Rilasci coordinati

Per ogni modifica bisogna aggiornare tre servizi nello stesso momento.

Tutti i costi dei microservizi, nessuno dei vantaggi.

L'errore opposto: i nanoservizi

Servizi così piccoli che gestirli costa più di quanto valgono.

Uno per tabella, uno per funzione: ogni operazione attraversa la rete cinque volte.

MICRO NON VUOL DIRE MINIMO

Abbastanza piccolo da essere posseduto da un team e cambiato in autonomia.

Verifichiamo: monolite e microservizi

1 Tre vantaggi del monolite che si perdono con i microservizi?

2 Perché un team di tre persone ne trae poco vantaggio?

3 Su quale asse dello Scale Cube stanno i microservizi?

4 Che cosa dice la legge di Conway?

Verifichiamo: confini e dati

5 Che cos'è un bounded context? Un esempio diverso da «prodotto».

6 Perché due servizi non devono condividere il database?

7 Che cos'è la consistenza eventuale?

8 Che cos'è un nanoservizio e perché è un problema?

Glossario

Termine	Significato
Accoppiamento	Quanto una parte dipende da un'altra. Tra servizi lo si vuole basso.
Coesione	Quanto sono legate le cose dentro un servizio. La si vuole alta.
Bounded context	Confine in cui ogni termine ha un solo significato e un solo modello.
Consistenza eventuale	Dati non coerenti in ogni istante, ma che lo diventano dopo un po'.
Deploy	Rilascio di una nuova versione in un ambiente.
Latenza	Tempo tra l'invio di una richiesta e l'arrivo della risposta.
Scalabilità orizzontale	Più carico aggiungendo macchine o copie del processo.
Transazione	Operazioni sui dati che riescono tutte o nessuna.

Che cos'è REST

REpresentational **S**tate **T**ransfer: uno stile architetturale per il web, descritto da Roy Fielding nella sua tesi di dottorato (2000). Non è una libreria né un protocollo: è un insieme di regole.

Risorse

Ogni cosa ha un indirizzo: /products/p-1

Interfaccia uniforme

Pochi verbi uguali per tutte: GET, POST, PUT, DELETE

Rappresentazioni

La risorsa viaggia come JSON, ma resta la stessa

Stateless

Ogni richiesta contiene tutto: il server non ricorda le precedenti

Client e server

Separati: evolvono in modo indipendente

Cacheable

La risposta dice se e quanto si può riutilizzare

Che cosa viaggia sulla rete

RICHIESTA

```
POST /products HTTP/1.1
Host: localhost:3001
Content-Type: application/json

{ "name": "Cuffie",
  "priceCents": 5990,
  "stock": 8 }
```

Prima riga: metodo e percorso, oppure codice di stato

RISPOSTA

```
HTTP/1.1 201 Created
Location: /products/p-4
Content-Type: application/json

{ "id": "p-4",
  "name": "Cuffie", ... }
```

Header: informazioni sul messaggio, una per riga

Corpo: i dati, dopo una riga vuota

Progettare bene un'API

Percorsi

Sì	No
GET /products	GET /getProducts
DELETE /products/p-1	POST /deleteProduct
GET /orders/o-1/lines	GET /orderLines?o=1
GET /products?q=mouse	GET /products/mouse

Sostantivi al plurale; il verbo è il metodo HTTP; la query string per filtri e ricerche.

Sicuri e idempotenti

Metodo	Sicuro	Idempotente
GET	sì	sì
PUT	no	sì
DELETE	no	sì
POST	no	no
PATCH	no	non garantito

Sicuro: non modifica nulla. **Idempotente:** ripeterlo dà lo stesso risultato della prima volta.

Ripasso: l'API REST di catalog

Le risorse hanno un indirizzo e si manipolano con i metodi HTTP.

Richiesta	Significato	Risposte
GET /products	Elenco	200
GET /products/p-1	Un prodotto	200, 404
POST /products	Crea	201, 400
PUT /products/p-1	Sostituisce	200, 400, 404
DELETE /products/p-1	Cancella	204, 404

I codici di stato che useremo

Codice	Nome	Quando
200	OK	Tutto bene, con un corpo
201	Created	Creato; Location dice dove
204	No Content	Tutto bene, senza corpo
400	Bad Request	Dati inviati non validi
404	Not Found	La risorsa non esiste
409	Conflict	Valida ma in conflitto (scorte finite)
500	Internal Server Error	Errore imprevisto nel server
501	Not Implemented	Non ancora fatto: i nostri TODO

Il nostro monolite e le sue cuciture

```
monolith/src/orders/routes.ts  
  
// CUCITURA 2  
for (const line of lines) {  
  const product = db.products  
    .find(p => p.id === line.productId!);  
  product.stock -= line.quantity;  
}
```

1. Gli ordini leggono i prodotti

direttamente dai dati del catalogo

2. Gli ordini modificano le scorte

dati che appartengono a un altro modulo

3. La notifica parte in modo sincrono

nello stesso processo dell'ordine

Laboratorio

30'

Esplora il monolite

Avvialo, prova `http/monolith.http`, trova le tre CUCITURE.

50'

Costruisci catalog

Completa i quattro TODO in `services/catalog/src/routes.ts`.

15'

Checkpoint

Lancia `npm test`: dieci test verdi.

```
cd microservizi/lezione-1/start/monolith → npm install → npm run dev
```

Il repo della lezione

github.com/mg-i-roc-maz/corsi

```
git clone https://github.com/
  mg-i-roc-maz/corsi
cd corsi/microservizi/lezione-1/start
code .
```

Già clonato? Dalla cartella corsi basta `git pull`.

microservizi/lezione-1/

Cartella	Contenuto
slides.pdf	Queste slide, in PDF
start/	Si lavora qui: monolite e scheletro di catalog
solution/	La soluzione, per riallinearsi
extra/	Test e soluzioni degli esercizi extra

0 · Verifica l'ambiente

0.1 Apri la cartella lezione-1/start in VS Code

Consigliato: aprila nel **Dev Container**, così Node 22 è già pronto. Come si fa: slide 0.4.

0.2 Nel terminale: **node -v**

Serve la versione 22 o successiva. Più vecchia o assente? Slide 0.5. Terminale integrato: Ctrl+ ` (Windows, Linux) o Cmd+ ` (Mac).

0.3 Apri **http/catalog.http**

Sopra ogni richiesta deve comparire **Send Request**: l'estensione REST Client è attiva.

0.4 · Il Dev Container

CHE COS'È

Un ambiente di sviluppo dentro un container Docker. VS Code si «sposta» dentro il container, dove Node 22 e le estensioni sono già installati.

Sul tuo computer servono solo **Docker Desktop** e VS Code con l'estensione **Dev Containers**. La configurazione è in `.devcontainer/devcontainer.json`.

- 1 Avvia Docker Desktop e aspetta che sia pronto
- 2 In VS Code apri la cartella lezione-1/start
- 3 In basso a destra compare l'avviso: clicca **Reopen in Container**. Se non compare: apri la palette dei comandi con **Cmd+Shift+P** (Mac) o **Ctrl+Shift+P** (Windows, Linux) e scegli «Dev Containers: Reopen in Container»
- 4 La prima volta servono alcuni minuti: scarica l'immagine e lancia npm install da solo
- 5 Sei dentro quando in basso a sinistra leggi **Dev Container: Corso microservizi**

```
node -v → v22.x cd monolith → npm run dev → http://localhost:3000
```

0.5 · Se node -v è sotto la 22

La via più semplice è il Dev Container (0.4). Se vuoi Node sul tuo computer, usa un **version manager**: installa più versioni e passi da una all'altra.

Mac e Linux · nvm

Installa nvm con il comando indicato su github.com/nvm-sh/nvm, riapri il terminale, poi:

```
nvm install 22  
nvm use 22  
nvm alias default 22
```

Su Mac, in alternativa: `brew install node@22`

Windows

Con nvm-windows (github.com/coreybutler/nvm-windows), da un terminale come amministratore:

```
nvm install 22  
nvm use 22
```

Oppure l'installer LTS da nodejs.org, o: `winget install OpenJS.NodeJS.LTS`

Poi chiudi e riapri il terminale e ricontrolla: `node -v` deve dare v22 o successiva.

1.1 · Avvia il monolite

```
cd monolith  
npm install  
npm run dev
```

npm install scarica le dipendenze, solo la prima volta. **npm run dev** avvia il server e lo riavvia a ogni salvataggio.

È pronto quando vedi:

```
"msg": "Server listening at  
http://127.0.0.1:3000"
```



Lascia questo terminale aperto e aprirne un **secondo** con il pulsante + per i comandi successivi.

1.2 · Prova le richieste

Apri **http/monolith.http** e clicca Send Request sopra ogni richiesta, nell'ordine.

Richiesta	Cosa aspettarti
GET /health	{"status":"ok","service":"monolith"}
GET /products	Tre prodotti; la tastiera ha stock 12
POST /orders	201, ordine o-1, totalCents 16970
GET /products	Tastiera a 11, mouse a 28: le scorte sono cambiate
POST /orders (100 monitor)	409, «Scorte insufficienti»
GET /orders	L'elenco con l'ordine creato

Nel primo terminale compare  Email di conferma a **mario@example.com**: la «notifica» del monolite.

1.3 · Rompi qualcosa

Aggiungi in fondo a `monolith.http` un ordine **senza items** e invialo:

```
### Ordine senza items
POST {{mono}}/orders
Content-Type: application/json

{
  "customerEmail": "test@example.com"
}
```

Risposta:

```
500 Internal Server Error
"message": "items is not iterable"
```

Fastify ha intercettato l'errore e il processo non è caduto. Ma il codice degli ordini si fida ciecamente dei dati in ingresso.

Nel servizio catalog vedremo come si evita: **validazione**.

1.4 · Leggi il codice, in quest'ordine

src/server.ts

Un processo, una porta, due moduli registrati

src/db.ts

Il «database» in memoria: **unico, condiviso da tutti**

src/catalog/routes.ts

Le rotte del catalogo

src/orders/routes.ts

Le rotte degli ordini: cerca i tre commenti **CUCITURA**

Fastify in cinque minuti

Un framework web per Node.js, progetto della OpenJS Foundation. Prende una richiesta HTTP, trova la funzione giusta e spedisce la risposta. Tutto il server sta in quattro righe:

```
import Fastify from "fastify";
const app = Fastify({ logger: true });
app.get("/health", async () => ({ status: "ok" }));
await app.listen({ port: 3001 });
```

Istanza

app: il server. In app.ts

Rotte

app.get, post, put, delete. In routes.ts

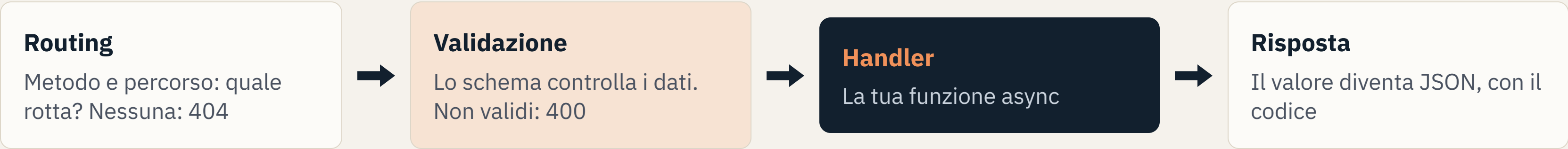
Schema

Regole JSON Schema sui dati. In schemas.ts

Plugin

Gruppi di rotte aggiunti con app.register

Il viaggio di una richiesta



Rispetto a Express	In Fastify
Validazione	Integrata, con JSON Schema
Log	Integrati (pino), in JSON
async / await	Basta un return: niente res.send obbligatorio
Test	app.inject() simula le richieste senza rete

1.5 · Compila RISPOSTE.md

Crea il file nella cartella start. Per ogni cucitura, una riga:

#	Che cosa fa	Dato toccato	Di chi è	Se catalog fosse separato?
1				
2				
3				

Domande. 1. Se gli ordini mandano in crash il processo, chi ne paga le conseguenze? 2. Quale cucitura sarà più difficile da separare, e perché? 3. Il 500 della richiesta 1.3: quale codice sarebbe stato corretto?

2.1 · Avvia il servizio catalog

Ferma il monolite con Ctrl+C. Il nuovo servizio è un'applicazione indipendente, sulla porta **3001**.

```
cd ../services/catalog  
npm install  
npm run dev
```

Con **http/catalog.http**: GET /health e GET /products funzionano già.

Prova GET /products/p-1:

```
501 Not Implemented  
{"error":"TODO 1: da implementare"}
```

È il tuo punto di partenza: quattro TODO da completare.

2.2 · Orientati nel codice

I file

File	Lo tocchi?
src/server.ts	No: avvio
src/app.ts	No: registra rotte
src/store.ts	No, ma leggilo
src/schemas.ts	No, ma leggilo
src/routes.ts	Sì: 4 TODO
test/products.test.ts	No: checkpoint

Le funzioni dello store

Funzione	Restituisce
list(q?)	Array di prodotti
get(id)	Prodotto o undefined
create(input)	Nuovo prodotto con id
replace(id, input)	Prodotto o undefined
remove(id)	true o false

2.3 · Anatomia di una rotta

```
app.get<{ Querystring: ListQuery }>("/products", async (request) => {  
  return store.list(request.query.q);  
});
```

`app.get("/products")`

Risponde alle GET su quel percorso.

`<{ Querystring }>`

Dice a TypeScript com'è fatta la query:
request.query.q è tipizzato.

`return ...`

Diventa il corpo JSON, con codice 200.

Per un codice diverso: `return reply.code(404).send({ error: "Prodotto non trovato" });`

TODO 1 · GET /products/:id

:id è un parametro del percorso: in /products/p-1 vale "p-1".

Si legge con `request.params.id`.

1. Cerca il prodotto con `store.get(...)`
2. Se non c'è: 404 con `{ error: "Prodotto non trovato" }`
3. Altrimenti restituisci il prodotto

Rinomina `_request` in `request`: il trattino basso indicava un parametro non usato.

Verifica con catalog.http

GET /products/p-1
→ 200, la tastiera

GET /products/p-999
→ 404, Prodotto non trovato

TODO 2 · POST /products

I dati arrivano nel corpo: `request.body`. La validazione è già fatta: se il corpo non rispetta lo schema, Fastify risponde 400 da solo.

1. Crea con `store.create(request.body)`
2. Rispondi 201 con il prodotto
3. Aggiungi l'header `Location: /products/<id>`

```
reply.code(201).header("Location", ...).send(...)
```

Verifica

POST cuffie

→ 201, location: /products/p-4

POST dati non validi

→ 400, ...required property 'stock'

Attenzione: a ogni salvataggio il server si riavvia e i dati in memoria tornano quelli iniziali.

TODO 3 · PUT /products/:id

PUT **sostituisce** l'intera risorsa: il client invia tutti i campi, non solo quelli che cambiano.

1. `store.replace(request.params.id, request.body)`
2. Se restituisce undefined: 404
3. Altrimenti il prodotto aggiornato

Verifica

PUT /products/p-2
→ 200, priceCents 3490

PUT /products/p-999
→ 404

La seconda richiesta aggiungila tu, copiando la prima.

TODO 4 · DELETE /products/:id

1. store.remove(request.params.id)
2. Se restituisce false: 404
3. Altrimenti 204 senza corpo

```
return reply.code(204).send();
```

Perché il secondo DELETE dà 404? La risorsa non c'è più.
Alcune API rispondono 204 anche la seconda volta:
entrambe le scelte sono legittime.

Verifica

DELETE /products/p-3
→ 204, corpo vuoto

DELETE /products/p-3 (di nuovo)
→ 404

GET /products/p-3
→ 404

3 · Checkpoint: npm test

Ferma il server e lancia **npm test**. Non serve il server acceso: i test simulano le richieste con `app.inject()`.

```
not ok 4 - GET /products/:id ...  
  Expected values to be strictly equal:  
    501 === 200
```

Si legge: ottenuto 501, atteso 200.

ALL'INIZIO

pass 4 · # fail 6

OBIETTIVO

pass 10 · # fail 0

Consegna: output di npm test con pass 10 e RISPOSTE.md compilato.

Problemi frequenti

Sintomo	Soluzione
node: command not found	Usa il Dev Container, o installa Node 22 e riapri il terminale
EADDRINUSE :::3001	C'è già un server acceso: Ctrl+C nell'altro terminale, o PORT=3002 npm run dev
Cannot find module 'fastify'	Manca npm install in quella cartella
Connection refused	Server spento o porta sbagliata: monolite 3000, catalog 3001
p-4 dà 404 dopo averlo creato	Il server si è riavviato al salvataggio: rifai la POST
Test 6: undefined !== '/products/p-4'	Manca l'header Location
Test 10: 200 !== 204	Usa reply.code(204).send() invece di return

ULTIMI 10 MINUTI

Cosa si rompe se...?

...il modulo ordini manda in crash il processo del monolite?

...catalog diventa un servizio separato, ma gli ordini modificano ancora le scorte?

...due persone ordinano l'ultimo monitor nello stesso istante?

PROSSIMA LEZIONE

Docker e Docker Compose

Il monolite diventa due servizi, ognuno nel suo container, con PostgreSQL al posto dei dati in memoria.

Per la volta prossima: Docker Desktop installato e `docker run hello-world` funzionante.

PER CHI FINISCE PRIMA, O PER CASA

Esercizi extra

#	Esercizio	Livello	Dove
1	Paginazione	base	catalog
2	Ordinamento	base	catalog
3	Aggiornamento parziale con PATCH	medio	catalog
4	Errori in formato uniforme	medio	catalog
5	Validazione degli ordini	medio	monolith
6	Trova i confini: un cinema	avanzato	carta e penna

Per gli esercizi 1–4 ci sono test pronti: copia [lezione-1/extra/extra.test.ts](#) in `services/catalog/test` e lancia `npm test`. Le soluzioni complete sono in `lezione-1/extra/soluzione`.

Paginazione

Con migliaia di prodotti, GET /products sarebbe enorme.
Aggiungi due parametri di query:

Parametro	Significato	Default, limiti
limit	Quanti restituirne	20, da 1 a 50
offset	Quanti saltarne	0, da 0 in su

E l'header **X-Total-Count** con il totale prima della
paginazione.

Suggerimenti: uno schema per la query string (default e type integer
convertono da soli); array.slice(inizio, fine).

Atteso

?limit=2

→ 2 prodotti, X-Total-Count: 3

?limit=2&offset=2

→ solo il terzo

?limit=500

→ 400

Per riflettere: e se tra pagina 1 e pagina 2 qualcuno cancella un
prodotto?

Ordinamento

Aggiungi a GET /products il parametro **sort**. Accetta solo:

Valore	Ordine
name · -name	Nome, A → Z o Z → A
price · -price	Prezzo, crescente o decrescente

Altri valori: 400. Con l'esercizio 1, si ordina **prima** di paginare.

Suggerimenti: enum nello schema; array.sort((a, b) => ...);
a.name.localeCompare(b.name, "it").

Atteso

?sort=-price
→ p-3, p-1, p-2

?sort=colore
→ 400

Aggiornamento parziale con PATCH

Con PUT si inviano tutti i campi anche per cambiare solo il prezzo. Aggiungi **PATCH /products/:id**, che aggiorna solo i campi presenti.

- Nuova funzione `store.patch(id, changes)` con lo spread `{ ...vecchio, ...modifiche }`
- Schema come `productInputSchema` ma senza campi obbligatori e con `minProperties: 1`
- Togli il default di `description`, o ogni PATCH la svuoterebbe
- Tipo del corpo: `Partial<ProductInput>`

Atteso

```
{ "priceCents": 7990 }  
→ 200, nome invariato  
{ }  
→ 400  
{ "priceCents": -1 }  
→ 400  
p-999  
→ 404
```

Errori in formato uniforme

Oggi l'API risponde agli errori in tre formati diversi. Per chi la usa, in particolare un altro servizio, ne serve **uno solo**: { error, details? }.

Situazione	Codice	Corpo
Rotta inesistente	404	<code>{"error": "Rotta GET /prodotti inesistente"}</code>
Dati non validi	400	<code>{"error": "Dati non validi", "details": [...]}</code>
Errore imprevisto	500	<code>{"error": "Errore interno"}</code>

Suggerimenti: in app.ts usa app.setNotFoundHandler e app.setErrorHandler<FastifyError>; error.validation è un array con instancePath e message.

Validazione degli ordini

Nel monolite, aggiungi uno schema a **POST /orders**:

- customerEmail obbligatoria, formato email
- items: array con almeno un elemento
- ogni elemento: productId stringa, quantity intero ≥ 1
- nessun campo in più

Prendi spunto da productInputSchema. Per l'email: format: "email".

Richiesta	Prima	Dopo
Senza items	500	400
quantity: 0	201 (!)	400
email "pippo"	201	400
Ordine corretto	201	201

Per riflettere: lo schema non può controllare che il prodotto esista. Perché?
E chi lo farà quando catalog sarà separato?

Trova i confini: un cinema

Gli utenti cercano un film, scelgono spettacolo e posto, pagano, ricevono il biglietto via email e all'ingresso mostrano un QR code. I gestori inseriscono film e orari, configurano le sale, vedono gli incassi. Chi accumula punti riceve sconti.

1 Elenca almeno quattro bounded context

2 Per ognuno: cosa possiede, cosa non fa

3 Due parole che cambiano significato

4 Frecce tra i contesti; tratteggiate se asincrone

Bonus: con un team di quattro persone, quali contesti terrestri insieme?

Per approfondire

J. Lewis, M. Fowler · *Microservices*

L'articolo che ha definito il termine (2014), su martinfowler.com

M. Fowler · *MonolithFirst, BoundedContext*

Due articoli brevi, sullo stesso sito

S. Newman · *Building Microservices*

O'Reilly, 2^a ed. 2021. E *Monolith to Microservices* (2019)

E. Evans · *Domain-Driven Design*

Addison-Wesley, 2003: da qui vengono i bounded context

M. Abbott, M. Fisher · *The Art of Scalability*

Addison-Wesley, 2009: lo Scale Cube
